REST ASSURED

CLASS 5
Complete Beginner-Friendly Notes

OAuth 2.0 token flow • Protected APIs • Employee CRUD • POJO classes • Serialization

Explanation of each important code block, realistic testing scenarios, corrected examples, practice exercises, troubleshooting, and industry best practices

Learning goal: Understand not only what to type, but why each line exists and how to test it safely.

Based on the supplied "Rest Assured class 5" training notes.

How to use this workbook

Read the concepts first, then type the code into your own Maven/TestNG project. Do not copy a complete test class without understanding its URL, payload, token response field names, and expected status codes. The examples below follow the local training setup in the source material, but the actual server contract is the final authority.

- **First pass:** understand the vocabulary and the token flow diagram.
- **Second pass:** run token generation by itself and inspect the response JSON.
- **Third pass:** call one protected Employee API using the access token.
- **Fourth pass:** add create, get, update, and delete assertions.
- **Final pass:** implement the POJO example and complete the exercises without viewing the solutions.

Important source-specific note: The uploaded class notes contain both snake_case token keys (access_token/refresh_token) and camelCase keys (accessToken/refreshToken) in different examples. They cannot be assumed interchangeable. Inspect the actual response from your local authorization server and use the exact key names it returns.

Contents

- 1. Understand the Class 5 workflow and local services
- 2. OAuth 2.0 fundamentals for beginners
- 3. Generate access and refresh tokens with Rest Assured
- 4. Use the access token to call protected Employee APIs
- 5. End-to-end employee CRUD workflow with assertions
- 6. Understand and fix the supplied UserApis code
- 7. POJO classes and Java-to-JSON serialization
- 8. Real-world scenarios and negative testing
- 9. Best practices for stable and secure API automation
- 10. Troubleshooting guide
- 11. Hands-on exercises with solutions
- 12. Quick reference and final checklist

1. Understand the Class 5 workflow

Your class covers two connected ideas: first, authenticate against an authorization server to obtain a token; second, present that token when calling an API that requires authorization. It also introduces POJO classes to build JSON request bodies using Java objects rather than maintaining many JSON files.

Service / concept	What it does in the training setup	Example
Authorization server	Checks the configured client/user credentials and issues tokens.	http://localhost:9080/oauth2/token
Resource server	Hosts APIs that require an access token.	http://localhost:9081/employees
Access token	Short-lived credential sent with each protected API request.	Authorization: Bearer <token>
Refresh token	Credential used by the configured refresh flow to request a new access token.	Depends on the local server endpoint and contract
POJO	A Java object representing request data.	EmployeeCreateRequest

Service / concept	What it does in the training setup	Example
Serialization	Converts that Java object into JSON for the HTTP request body.	request.body(employee)

The complete request flow

Step	Action	Expected purpose
1	POST /oauth2/token to port 9080 with form parameters	Receive access and refresh tokens
2	Read token values from the response JSON	Store tokens in test-class fields
3	GET or POST /employees on port 9081 with the access token	Authenticate to the protected API
4	Capture the created employee ID	Reuse it for GET, PUT and DELETE
5	Assert the response status and important body values	Prove the API behavior is correct
6	Clean up test-created data	Prevent repeated test runs from polluting the environment

Real-world analogy: The access token is like a temporary entry pass. The authorization server issues the pass; the resource server checks it before allowing an employee operation. A valid pass does not automatically mean the caller has permission for every action.

2. OAuth 2.0 fundamentals for beginners

Authentication vs authorization

Authentication answers “Who are you?” Authorization answers “What are you allowed to do?” In this training flow, the token proves a successful authentication context. The resource server can still reject the request if the token is invalid, expired, or lacks the required scope/permission.

Term	Plain-language meaning	Testing implication
Client ID	Identifier for the software/application requesting a token.	Wrong ID may prevent token generation.
Client secret	Confidential credential belonging to the client application.	Never commit it to source control or print it in logs.
Username/password	User credentials used by the grant configured in this local demo.	Negative tests should cover invalid credentials.
Grant type	The type of process used to obtain a token.	The authorization server must support the requested value.
Scope	The access level requested, such as read/write.	A token with only read permission may not create or update records.
Access token	Credential sent to the protected resource API.	Use as a Bearer token; validate protected endpoints reject missing/invalid tokens.

Term	Plain-language meaning	Testing implication
Refresh token	Credential for requesting a replacement access token where supported.	Test the specific refresh endpoint and its expected response shape.
401 Unauthorized	The request is not successfully authenticated.	Try no token, invalid token, or expired token.
403 Forbidden	Authentication may have succeeded, but the action is not permitted.	Try a valid token lacking the required role/scope.

Security warning: The source uses grant_type=password for a local learning exercise. Do not treat this as a production recommendation. OAuth security guidance discourages the Resource Owner Password Credentials grant for new deployments. Use the flow designed for your application (commonly Authorization Code with PKCE for user sign-in, or Client Credentials for service-to-service access).

Where the request goes

```
Rest Assured test
    |
    | POST /oauth2/token (form-encoded credentials)
    v
Authorization server :9080
    |
    | access token + refresh token
    v
Rest Assured test stores token
    |
    | Authorization: Bearer <access-token>
    v
Resource server :9081 /employees
    |
    v
Response + assertions
```

3. Generate access and refresh tokens with Rest Assured

3.1 Why form-urlencoded?

Your source uses `application/x-www-form-urlencoded` for the token endpoint. This means the request body is sent as encoded key/value pairs rather than a JSON object. Rest Assured's `formParam()` method builds these fields and encodes them for the request.

```
Content-Type: application/x-www-form-urlencoded

grant_type=password&client_id=employee-client&client_secret=...&username=Admin&password=...&scope=read%20write
```

Do not send the token request as JSON unless your local server explicitly expects JSON. The request format is part of the endpoint contract.

3.2 Understand each form field

Field	What it means	Common mistake
grant_type	Selects the supported token grant.	Using a grant that the server/client has

Field	What it means	Common mistake
		not enabled.
client_id	Identifies the OAuth client registered on the authorization server.	Using the wrong client ID.
client_secret	Verifies the confidential client when required.	Hardcoding or logging the secret.
username	User name for the local password-grant exercise.	Using a user that does not exist or is disabled.
password	Password for that training user.	Sending an incorrect password or exposing it in logs.
scope	Access level requested by the client.	The supplied notes have a typo `scrope`; the correct parameter spelling is `scope`.

3.3 A corrected basic token request

This is a teaching version of the request. Replace credentials with your local test credentials. The sample status code is 200 because token endpoints commonly return success this way, but check your server contract rather than assuming it.

```
import static io.restassured.RestAssured.given;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

import io.restassured.http.ContentType;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;

Response tokenResponse = given()
    .contentType(ContentType.URLENC)
    .formParam("grant_type", "password")
    .formParam("client_id", "employee-client")
    .formParam("client_secret", "employee-secret")
    .formParam("username", "Admin")
    .formParam("password", "admin123")
    .formParam("scope", "read write")
.when()
    .post("http://localhost:9080/oauth2/token");

assertThat(tokenResponse.statusCode(), equalTo(200));
assertThat(tokenResponse.time(), lessThan(5000L));

JsonPath tokenJson = tokenResponse.jsonPath();
String accessToken = tokenJson.getString("access_token");
String refreshToken = tokenJson.getString("refresh_token");

assertThat("access token should be present", accessToken, not(isEmptyOrNullString()));
assertThat("refresh token should be present", refreshToken, not(isEmptyOrNullString()));
```

Line-by-line explanation

- **`given()`** starts the request specification where headers, authentication, query parameters, and body data are configured.
- **`contentType(ContentType.URLENC)`** sets the form URL-encoded content type expected by the training token endpoint.

- **`formParam(...)`** adds one field to the form body. The `scope` line must be spelled exactly.
- **`post(...)`** sends an HTTP POST request to the authorization server.
- **`statusCode()`** reads the actual HTTP status code. The assertion makes a failing token response fail the test.
- **`time()`** measures response duration in milliseconds. A 5-second limit is only an example threshold; align it to your test environment and performance objective.
- **`jsonPath()`** provides convenient access to values inside the JSON response body.
- **`getString(...)`** extracts a named field. JSON key spelling and capitalization must match the actual response.
- **`not(isEmptyOrNullString())`** ensures a token was actually returned rather than silently continuing with null/empty data.

Critical field-name mismatch in the source: One part of the uploaded notes reads `access_token` and `refresh_token`; the TestNG class reads `accessToken` and `refreshToken`. Inspect `tokenResponse.asPrettyString()` locally to see what your app returns, then use those exact names. Do not guess. If the token body contains an error object, assert/report that before trying to call the Employee API.

3.4 A reusable token helper

As the test suite grows, place token generation in a helper method instead of copying the same token request into every test class. This example is deliberately simple and shows one way to pass credentials from environment variables.

```
public class OAuthTokenClient {
    private static final String TOKEN_URL =
        "http://localhost:9080/oauth2/token";

    public static String getAccessToken() {
        String clientId = System.getenv("OAUTH_CLIENT_ID");
        String clientSecret = System.getenv("OAUTH_CLIENT_SECRET");
        String username = System.getenv("API_TEST_USERNAME");
        String password = System.getenv("API_TEST_PASSWORD");

        if (clientId == null || clientSecret == null ||
            username == null || password == null) {
            throw new IllegalStateException(
                "Set OAuth test credentials as environment variables.");
        }

        Response response = RestAssured.given()
            .contentType(ContentType.URLENC)
            .formParam("grant_type", "password")
            .formParam("client_id", clientId)
            .formParam("client_secret", clientSecret)
            .formParam("username", username)
            .formParam("password", password)
            .formParam("scope", "read write")
            .post(TOKEN_URL);

        response.then().statusCode(200);
        String token = response.jsonPath().getString("access_token");
        if (token == null || token.isBlank()) {
            throw new AssertionError("Token response did not include access_token");
        }
        return token;
    }
}
```

This helper uses the snake_case field `access_token`. Change it only if inspection confirms that the training server returns a different key. A reusable helper makes credentials and token handling consistent, but do not print the response body in CI if it contains live credentials or tokens.

4. Use the access token to call protected Employee APIs

4.1 Send token as Bearer authentication

The token endpoint and employee endpoint are different services. Obtaining a token does not automatically attach it to future requests. Every protected request must send the access token in the expected authorization header.

```
Response response = given()
    .auth().oauth2(accessToken)
    .contentType(ContentType.JSON)
    .body(employeePayload)
.when()
    .post("http://localhost:9081/employees");

response.then()
    .statusCode(201)
    .contentType(ContentType.JSON);
```

Important correction: Use `auth().oauth2(accessToken)` for an OAuth 2.0 Bearer token. `auth().oauth(...)` refers to OAuth 1-style authentication in Rest Assured and is not the correct method for the Bearer-token flow in these notes.

4.2 Example create-employee payload

```
{
  "name": "Suraj",
  "email": "suraj3@gmail.com",
  "department": "Testing3",
  "salary": 230000,
  "createdAt": "2026-09-25T01:57:05.209Z"
}
```

This payload matches the shape shown in the uploaded class notes for the API on port 9081. Confirm whether `createdAt` should be sent by the client or generated by the server; many APIs own audit timestamps themselves. Do not assume every API expects client-supplied timestamps.

4.3 Read the created employee ID

```
Response createResponse = given()
    .auth().oauth2(accessToken)
    .contentType(ContentType.JSON)
    .body(employeePayload)
.when()
    .post("http://localhost:9081/employees");

createResponse.then().statusCode(201);
int employeeId = createResponse.jsonPath().getInt("id");
assertThat(employeeId, greaterThan(0));
```

The API assigns an identifier when it creates the employee. Capturing that ID is more reliable than assuming the ID will always be 1. The exact success code could be 200 or 201 depending on the documented API contract; 201 is commonly used when a new resource is created.

4.4 Path parameter

```
Response getResponse = given()
    .auth().oauth2(accessToken)
    .pathParam("id", employeeId)
.when()
    .get("http://localhost:9081/employees/{id}");

getResponse.then().statusCode(200);
assertThat(getResponse.jsonPath().getInt("id"), equalTo(employeeId));
```

`{id}` is a placeholder in the URL. `pathParam("id", employeeId)` tells Rest Assured which value to put there. This is cleaner and safer than manually concatenating strings such as `"/employees/" + employeeId`.

4.5 Update and delete

```
// PUT: send a full replacement representation, if that is the API contract.
Response updateResponse = given()
    .auth().oauth2(accessToken)
    .contentType(ContentType.JSON)
    .pathParam("id", employeeId)
    .body(updatedEmployeePayload)
.when()
    .put("http://localhost:9081/employees/{id}");

updateResponse.then().statusCode(200); // Check your API's actual contract.

// DELETE: assert the contractually expected response.
Response deleteResponse = given()
    .auth().oauth2(accessToken)
    .pathParam("id", employeeId)
.when()
    .delete("http://localhost:9081/employees/{id}");

assertThat(deleteResponse.statusCode(), anyOf(is(200), is(202), is(204)));
```

A permissive `anyOf` may be helpful while exploring an unfamiliar training API, but in a final regression test use the single status code defined by the API contract. A 204 No Content response usually has no response body; do not parse JSON from it unless the server actually returns a body.

5. End-to-end Employee CRUD workflow

CRUD means Create, Read, Update, Delete. For a beginner, first get the whole sequence working with one created record. Then split it into independently runnable test cases and use setup/cleanup methods or a dependable test-data strategy.

Operation	HTTP method	Example endpoint	Typical success status (verify contract)
Create	POST	/employees	201 Created
Read one	GET	/employees/{id}	200 OK
Update	PUT	/employees/{id}	200 OK or 204 No Content
Delete	DELETE	/employees/{id}	204 No Content or documented alternative
Read after delete	GET	/employees/{id}	404 Not Found, if deleted records are no longer visible

Beginner-friendly end-to-end test sketch

```
public class EmployeeCrudTest {
    private String accessToken;
    private int employeeId;

    @BeforeClass
    public void setUp() {
        accessToken = OAuthTokenClient.getAccessToken();
    }

    @Test
    public void employeeCrudWorkflow() {
        // 1. CREATE
        Map<String, Object> newEmployee = new HashMap<>();
        newEmployee.put("name", "Api Test Employee");
        newEmployee.put("email", "api-test-" + System.nanoTime() + "@example.test");
        newEmployee.put("department", "QA");
        newEmployee.put("salary", 90000);

        Response create = given()
            .auth().oauth2(accessToken)
            .contentType(ContentType.JSON)
            .body(newEmployee)
        .when().post("http://localhost:9081/employees");

        create.then().statusCode(201);
        employeeId = create.jsonPath().getInt("id");
        assertThat(employeeId, greaterThan(0));

        // 2. READ
        Response get = given()
            .auth().oauth2(accessToken)
            .pathParam("id", employeeId)
        .when().get("http://localhost:9081/employees/{id}");

        get.then().statusCode(200);
        assertThat(get.jsonPath().getString("name"), equalTo("Api Test Employee"));

        // 3. UPDATE (send all required PUT fields)
        newEmployee.put("name", "Api Test Employee Updated");
        Response update = given()
            .auth().oauth2(accessToken)
            .contentType(ContentType.JSON)
            .pathParam("id", employeeId)
            .body(newEmployee)
        .when().put("http://localhost:9081/employees/{id}");

        update.then().statusCode(200); // Adapt to server contract.

        // Verify the update persisted.
        Response verify = given()
            .auth().oauth2(accessToken)
            .pathParam("id", employeeId)
        .when().get("http://localhost:9081/employees/{id}");

        verify.then().statusCode(200);
        assertThat(verify.jsonPath().getString("name"),
                equalTo("Api Test Employee Updated"));

        // 4. DELETE
        Response delete = given()
            .auth().oauth2(accessToken)
            .pathParam("id", employeeId)
        .when().delete("http://localhost:9081/employees/{id}");

        assertThat(delete.statusCode(), anyOf(is(200), is(202), is(204)));
```

```
  }
}
```

Before using this sketch: Add the required imports (`java.util.*`, TestNG, Rest Assured, Response, ContentType, and Hamcrest). Verify which fields are mandatory. Replace broad status assertions with the exact contract for your API. Add cleanup in a `finally`/`@AfterMethod` strategy if test failures must not leave data behind.

Why the source uses `@BeforeClass` and `@Test(priority=...)`

`@BeforeClass` runs setup once before test methods in the class, so token generation can happen once. `@Test(priority=...)` sorts tests by priority, and the source relies on an ID created by an earlier test. That works for a simple classroom demonstration, but it can be brittle when a test is run alone, reordered, retried, or parallelized.

- **Beginner exercise:** keep the CRUD workflow in one test until you understand the data dependency.
- **Next step:** use explicit TestNG `dependsOnMethods` only when a dependent workflow is intentional.
- **Professional approach:** make tests independently runnable where possible, create their own data, and clean up their own resources.

6. Understand and fix the supplied UserApis code

The uploaded source has a `UserApis` class with fields `accessToken`, `refreshToken`, and `employeeId`. Its intended sequence is: get a token in `@BeforeClass`, create an employee, get it by ID, update it, then delete it.

Source detail	Why it matters	Recommended action
`jsonPath.getString("accessToken")`	May not match the token server response key.	Inspect response JSON. Use the exact returned field, e.g. access_token or accessToken.
Token request logs all	Can expose password, client secret, or issued tokens.	Avoid logging sensitive request fields and token response bodies.
No status assertions before reading token	The code may continue after a failed token request and use a null token.	Assert expected token status and token is non-empty first.
`employeeId` assigned without asserting create response	A failed create could lead to ID 0/null and misleading later failures.	Assert create status and validate `id` before using it.
Tests use priority	ID dependency can make tests fragile or fail when run individually.	Use reliable setup/data dependencies or one workflow test first.
Request/response logging	Large logs slow diagnosis and can leak PII or credentials.	Log only necessary safe details; redact Authorization and secrets.
Fixed payload path strings	Hard-coded separators and file names can break across machines.	Use `Paths.get(...)` and verify fixture files exist.

A safer setUp pattern

```
@BeforeClass
public void setUp() {
   accessToken = OAuthTokenClient.getAccessToken();
   Assert.assertNotNull(accessToken, "Access token must be available before tests run");
   Assert.assertFalse(accessToken.isBlank(), "Access token must not be empty");
}
```

The reusable helper should itself assert that token generation succeeded. The extra assertions in setup make the class fail early with a clear message rather than creating multiple confusing downstream failures.

Portable file paths

```
Path payloadPath = Paths.get(
    System.getProperty("user.dir"),
    "src", "test", "resources", "payload", "createEmployeePayload.json");

if (!Files.exists(payloadPath)) {
    throw new IllegalStateException("Missing payload file: " + payloadPath);
}

File payloadFile = payloadPath.toFile();
```

This avoids manually typing `//` or Windows backslashes. It also fails early if a fixture is missing.

7. POJO classes and Java-to-JSON serialization

7.1 What is a POJO?

POJO stands for Plain Old Java Object. It is a regular Java class used to hold data. In API testing, a POJO can represent the JSON body expected by an endpoint. You set the fields in Java, pass the object to Rest Assured, and a configured serializer converts it to JSON.

Term	Beginner explanation	Example
Field	A variable stored inside the object.	`private String name;`
Private	The field is hidden from direct access outside the class.	`private String email;`
Setter	A method to assign a value to a field.	`setName("Vikas")`
Getter	A method to read a field value.	`getName()`
Serialization	Convert Java object into JSON for the request body.	`given().body(employee)`
Deserialization	Convert JSON response data into a Java object.	`response.as(EmployeeResponse.class)`

Keep the models separate: The source contains two distinct employee API examples: port 9081 uses fields like name/email/department/salary/createdAt, while port 9196 uses firstName/lastName/address/mobileNumber/country/email. Do not reuse one POJO across both APIs unless the contracts truly match.

7.2 A corrected POJO for the port 9196 example

```
package TestLayer2;

public class Employee {
    private String firstName;
    private String lastName;
    private String mobileNumber;
    private String address;
    private String email;
    private String country;
```

```
    public Employee() {
        // Default constructor, useful for many serializers.
    }

    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }

    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }

    public String getMobileNumber() { return mobileNumber; }
    public void setMobileNumber(String mobileNumber) { this.mobileNumber = mobileNumber; }

    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getCountry() { return country; }
    public void setCountry(String country) { this.country = country; }
}
```

Each field is private, which is a standard encapsulation practice. Getters and setters provide controlled access. The mobile number is stored as a String because phone numbers are identifiers, not quantities to add or multiply; leading zeroes must be preserved.

7.3 Use the POJO as the request body

```
Employee employee = new Employee();
employee.setFirstName("Vikas");
employee.setLastName("Mule");
employee.setAddress("Mumbai");
employee.setCountry("India");
employee.setEmail("vikas@example.test");
employee.setMobileNumber("090909011");

Response response = given()
    .auth().basic("Admin", "admin123") // Local example only; avoid hardcoded secrets.
    .contentType(ContentType.JSON)
    .body(employee)
.when()
    .post("http://localhost:9196/api/employees");

response.then().log().ifValidationFails();
response.then().statusCode(201); // Verify the API's documented success code.
```

The spelling error `enp.setEmail(...)` in the original sample is a compile-time error; it should be `emp.setEmail(...)` or use the variable name you actually declared. The request content type is set to JSON because the object will be serialized to JSON.

7.4 Serialization explained

```
Java object (before sending)
Employee {
  firstName = "Vikas",
  lastName = "Mule",
  country = "India"
}
```

```
JSON sent over HTTP
{
  "firstName": "Vikas",
  "lastName": "Mule",
  "country": "India"
}
```

The serializer (for example Jackson, when configured on the classpath) performs the conversion. If Rest Assured reports it cannot serialize the object, check that an appropriate JSON mapper dependency is available and compatible with your project. Avoid manually writing JSON and a POJO for the same payload unless there is a clear reason.

7.5 A separate POJO for the port 9081 employee shape

```
public class EmployeeCreateRequest {
    private String name;
    private String email;
    private String department;
    private Integer salary;
    private String createdAt;

    public EmployeeCreateRequest() { }

    public EmployeeCreateRequest(String name, String email,
            String department, Integer salary, String createdAt) {
        this.name = name;
        this.email = email;
        this.department = department;
        this.salary = salary;
        this.createdAt = createdAt;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public Integer getSalary() { return salary; }
    public void setSalary(Integer salary) { this.salary = salary; }
    public String getCreatedAt() { return createdAt; }
    public void setCreatedAt(String createdAt) { this.createdAt = createdAt; }
}
```

Only include `createdAt` if the API contract expects the client to supply it. A better production design often lets the server assign creation time, but the supplied notes include it in the example payload.

8. Real-world scenarios and negative testing

Good API automation tests both expected success and important failure behavior. Test failures should make it clear whether the API is behaving incorrectly or the request itself is invalid.

Scenario	Request variation	Expected result to verify
Valid access token	Call GET /employees with a fresh token.	Documented success status and valid employee/list response.
No access token	Call a protected endpoint without Authorization.	Usually 401; check server contract.
Malformed/invalid token	Send `Bearer abc-not-a-token`.	Usually 401; no protected data should be returned.

Scenario	Request variation	Expected result to verify
Expired token	Call the API using an expired token.	Usually 401; then test the refresh endpoint if supported.
Insufficient scope/role	Use a valid token lacking write permission.	Usually 403; verify no write happened.
Invalid employee body	Omit a required field or send salary with the wrong type.	Often 400 or 422; verify useful validation information.
Duplicate email	Create a second employee using an existing email.	Conflict/validation result based on contract; no duplicate record.
Unknown employee ID	GET /employees/999999.	Usually 404.
Update non-existent employee	PUT an ID that does not exist.	Usually 404 or a documented upsert response; do not assume.
Delete twice	Delete the same employee twice.	First delete succeeds; second request follows documented not-found/idempotency behavior.
Token endpoint invalid credentials	Wrong client secret or user password.	Token request fails; no usable access token is returned.
Serialization mapping bug	Send POJO with mismatched field names.	Request JSON should match the contract; server validation should be clear.

Example: verify a request is rejected without a token

```
Response response = given()
   .get("http://localhost:9081/employees");

assertThat(response.statusCode(), equalTo(401));
```

Do not set `.auth().oauth2(accessToken)` in this specific test, because the purpose is to prove the API rejects an unauthenticated request. Keep the test separate from a positive test that uses a valid token.

Example: verify a bad token is rejected

```
Response response = given()
   .header("Authorization", "Bearer invalid-token-value")
   .get("http://localhost:9081/employees");

assertThat(response.statusCode(), equalTo(401));
```

Example: validate created employee data

```
createResponse.then()
   .statusCode(201)
   .contentType(ContentType.JSON);

JsonPath json = createResponse.jsonPath();
assertThat(json.getInt("id"), greaterThan(0));
assertThat(json.getString("name"), equalTo("Api Test Employee"));
assertThat(json.getString("email"), containsString("@"));
assertThat(json.getString("department"), equalTo("QA"));
```

The `containsString("@")` check is only a small smoke check, not a complete email validation rule. Prefer verifying the value stored by the service matches the value submitted, and use schema/contract validation where appropriate.

9. Best practices for secure, stable Rest Assured tests

- **Never log secrets or tokens.** Request logging can print client secrets, passwords, Authorization headers, and personally identifiable information. Use masked/redacted logs and only log safe response details.
- **Do not hardcode secrets in source code.** Use environment variables or a secret store in CI. The literal credentials in this workbook are copied from the local classroom example and should only be used in a disposable local environment.
- **Assert before extracting dependent data.** Check token generation status and non-empty token before using it; check create status and positive ID before the GET/PUT/DELETE sequence.
- **Use the right authentication method.** For a Bearer OAuth 2.0 access token, use `auth().oauth2(token)`. Do not confuse it with Basic auth or OAuth 1.
- **Verify the exact JSON response fields.** Token servers differ in response naming. Inspect the actual response and use the exact snake_case or camelCase key.
- **Keep API models contract-specific.** Do not reuse a POJO whose fields do not match the endpoint schema.
- **Use independent test data.** Avoid fixed employee IDs and fixed email addresses. Generate unique test values and clean up resources created by tests.
- **Prefer reliable test structure.** Avoid hidden dependence on test priority. Make dependencies explicit and avoid parallel execution when tests mutate the same record.
- **Use meaningful assertions.** Verify status, key response fields, state changes, and error behavior—not only that a response was printed.
- **Avoid brittle response-time limits.** Use agreed performance thresholds and controlled test environments; general functional tests should not fail randomly due to noisy timing.
- **Use path parameters.** Prefer `.pathParam("id", employeeId)` with `{id}` in the URL.
- **Use `log().ifValidationFails()` for diagnostics.** It reduces routine log noise, but first ensure the logging configuration does not expose secrets.
- **Make payload files portable.** Use `Paths.get(...)` or classpath resources, not hardcoded machine-specific paths.
- **Match expected status codes to the API contract.** 201 vs 200, PUT response semantics, DELETE response codes, and error codes vary. Do not use broad status assertions in final regression tests.
- **Separate test and production environments.** Never point destructive CRUD tests at production systems or real employee data.

A reusable request specification

```
RequestSpecification baseRequest = new RequestSpecBuilder()
   .setBaseUri("http://localhost:9081")
   .setContentType(ContentType.JSON)
   .build();

Response response = given()
   .spec(baseRequest)
   .auth().oauth2(accessToken)
.when()
   .get("/employees");
```

A shared request specification centralizes common settings, reducing repeated code. Do not put one user-specific or mutable token into a global shared specification if tests run concurrently; attach the token safely per test/request.

10. Troubleshooting guide

Symptom	Likely causes	What to check
Token endpoint returns 400	Missing/wrong form fields, typo, unsupported grant, scope not accepted.	Check Content-Type, exact `scope` spelling, request fields, server logs.
Token endpoint returns 401	Wrong client credentials or user credentials.	Verify configured client ID/secret, username/password, and whether the client supports the grant.
Access token is null	Wrong JSON key or token request failed.	Print a sanitized body locally; assert status before extracting; inspect exact field names.
Protected API returns 401	No token, bad token, expired token, wrong auth scheme.	Use `.auth().oauth2(accessToken)`; ensure token is from correct authorization server.
Protected API returns 403	Token valid but insufficient scope/role/permission.	Check requested scopes and user/client role configuration.
POST returns 415	Wrong Content-Type.	Use JSON content type for JSON request body; use URLENC for token endpoint.
POST returns 400/422	JSON fields missing or wrong type.	Compare request body with endpoint contract and server validation message.
Employee ID is 0	Create response failed or response key is different.	Assert create status first and inspect response JSON before extracting ID.
Tests pass alone but fail as suite	Test order, shared fixed data, token expiry, cleanup, parallel execution.	Remove hidden ordering and shared mutable state; isolate data and refresh tokens as needed.
POJO is not serialized	Missing mapper dependency or incorrect object getters/field mapping.	Check configured JSON mapper and naming conventions; inspect safe request JSON.
Wrong payload file / FileNotFoundException	Hard-coded path or incorrect working directory.	Build path using `Paths.get`; assert file exists before request.

11. Hands-on exercises with solutions

Exercise 1 - Generate a token

Write a POST request to `/oauth2/token` using form-url-encoded fields. Assert a successful status, then extract the access token and confirm it is not null or blank.

Solution outline

```
Response response = given()
   .contentType(ContentType.URLENC)
   .formParam("grant_type", "password")
   .formParam("client_id", clientId)
   .formParam("client_secret", clientSecret)
   .formParam("username", username)
   .formParam("password", password)
   .formParam("scope", "read write")
   .post("http://localhost:9080/oauth2/token");
```

```
response.then().statusCode(200);
String token = response.jsonPath().getString("access_token");
assertThat(token, not(isEmptyOrNullString()));
```

Exercise 2 - Correct the token typo

Given the source line `.formParam("scrope", "read write")`, correct it and explain why the typo matters.

Solution

```
.formParam("scope", "read write")
```

HTTP form field names are part of the endpoint contract. If the server expects `scope`, sending `scrope` does not request that scope and may cause a validation error or a token with unexpected privileges.

Exercise 3 - Call GET /employees

Use a valid token to GET the employee collection and assert the expected success status.

Solution

```
Response response = given()
    .auth().oauth2(accessToken)
.when()
    .get("http://localhost:9081/employees");

response.then().statusCode(200);
```

Exercise 4 - Test without authentication

Call the same endpoint without adding the access token. What should a protected API typically return?

Solution

```
Response response = given()
    .get("http://localhost:9081/employees");
assertThat(response.statusCode(), equalTo(401));
```

This is a negative test. Confirm the expected code in the local API contract.

Exercise 5 - Capture the created employee ID

A successful create response contains an `id`. Extract it and verify it is greater than zero.

Solution

```
createResponse.then().statusCode(201);
int employeeId = createResponse.jsonPath().getInt("id");
assertThat(employeeId, greaterThan(0));
```

Exercise 6 - Use a path parameter

Fetch the employee whose ID is stored in `employeeId`.

Solution

```
Response response = given()
    .auth().oauth2(accessToken)
    .pathParam("id", employeeId)
    .get("http://localhost:9081/employees/{id}");
response.then().statusCode(200);
```

Exercise 7 - Fix wrong Rest Assured authentication

The code uses `.auth().oauth(accessToken)` but the value is an OAuth 2.0 Bearer token. Correct it.

Solution

```
.auth().oauth2(accessToken)
```

Exercise 8 - Fix the POJO typo

The source declares `Employee emp = new Employee();` but calls `enp.setEmail(...)`. Correct the variable name.

Solution

```
emp.setEmail("vikas@example.test");
```

Exercise 9 - Model salary with an appropriate type

The API examples show salary as a numeric JSON value. Choose a type and state one consideration.

Solution

Use an integer type such as `Integer` or `Long` if salary is defined in whole currency units and fits the range; use `BigDecimal` for precise decimal monetary values. Match the API schema and business rules. Avoid using a floating-point type for money when exact decimal behavior is needed.

Exercise 10 - Plan a refresh-token test

Describe the test sequence without assuming your server's exact refresh endpoint contract.

Solution outline

1. Obtain the original access token and refresh token using the documented token endpoint.
2. Call a protected API to prove the access token works.
3. Use the documented refresh endpoint and request format to exchange the refresh token.
4. Assert the documented success code and inspect the response for the new access token.
5. Call the protected API using the new token.
6. Test invalid/revoked refresh token behavior and avoid logging either token.

Exercise 11 - Find four issues in the original code

Look for: token JSON key mismatch, sensitive logging, missing status assertions, fixed test-order dependency, or non-portable file paths.

Solution

Any four of these are valid: (1) token key names vary between `access_token` and `accessToken`; (2) `request.log().all()` can reveal credentials; (3) token and create responses are read before status is asserted; (4) tests rely on `priority` and a shared employee ID; (5) path strings are manually assembled with platform-specific separators; (6) the source's initial overview mentions `oauth(accessToken)` even though OAuth 2.0 Bearer usage should use `oauth2(accessToken)`.

Exercise 12 - Mini capstone

Create a TestNG workflow that obtains a token, creates an employee with a unique email, verifies the returned ID and name, updates the name, verifies the update through GET, and deletes the created record. Add one negative test proving that the collection endpoint rejects a request with no token. Use safe logging and cleanup.

Capstone acceptance checklist: Token is non-empty; each response status is asserted; the created ID is captured rather than hardcoded; updated state is verified via GET; delete response matches contract; unauthenticated request is rejected; secrets/tokens are not exposed in logs.

12. Quick reference

Task	Rest Assured pattern
Create request spec	`given()`
Form URL-encoded body	`contentType(ContentType.URLENC).formParam(...)`
JSON request	`contentType(ContentType.JSON).body(payload)`
OAuth 2.0 Bearer token	`auth().oauth2(accessToken)`
Basic auth for the separate port 9196 example	`auth().basic(username, password)`
POST	`.post(url)`
GET	`.get(url)`
PUT	`.put(url)`
PATCH	`.patch(url)`
DELETE	`.delete(url)`
Path parameter	`.pathParam("id", id)` with `/{id}` in URL
JSON response path	`response.jsonPath().getString("field")`
Pretty response body	`response.body().asPrettyString()` - use carefully if it may contain secrets
Check a status code	`response.then().statusCode(200)`
Use a POJO body	`.body(employeePojo)` when a JSON mapper is configured
Log only on validation failure	`.then().log().ifValidationFails()`

Final checklist before you run the tests

- Both local applications are running: authorization server on port 9080 and resource server on port 9081; port 9196 is a separate API example.
- The client and user credentials match the configuration of the local authorization server.
- The token request uses form URL encoding and the exact `scope` spelling.
- The token response keys are confirmed from the actual server response.
- Protected APIs use `.auth().oauth2(accessToken)` and not the OAuth 1 method.
- Create response status and ID are asserted before the ID is reused.
- Update and delete expected status codes match the API contract.
- Payload names match the endpoint's contract; the two Employee APIs are not assumed to share one schema.
- No real secrets, passwords, tokens, or sensitive employee data are committed or logged.
- Tests use unique records and clean up created data safely.

Final takeaway: OAuth 2.0 testing is not just "get a token and paste it". A reliable test verifies token generation, uses the correct Bearer scheme, checks authorization failures, validates response data, isolates

test records, and protects credentials. POJOs can make request payloads easier to maintain when their fields accurately represent the API contract.